

Rozwiązanie: Serwer Przetwarzania Obrazów (Box Blur)

Oto rozwiązanie zadania na podstawie dostarczonego pliku `k2_2023.pdf` [cite: 57, 59]. Program stanowi serwer [cite: 66, 68], który łączy się sekwencyjnie z klientami, odbiera obraz PNG [cite: 74], zapisuje go z unikalnym znacznikiem czasu [cite: 78, 79], zrównolegla operację nałożenia algorytmu Box Blur wykorzystując interfejs graficzny [cite: 83, 85, 93], zapisuje historię do bazy danych SQLite [cite: 95, 96, 97], po czym zwraca przetworzony obraz do klienta [cite: 104, 105].

Wymagania wstępne

- JDK (Java Development Kit) w wersji 11 lub nowszej z podpiętą biblioteką **JavaFX**.
- Sterownik **SQLite JDBC** dodany do classpath projektu (np. `sqlite-jdbc.jar`).

Kod źródłowy serwera (BlurServer.java)

```
import javafx.application.Application;
import javafx.application.Platform;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.control.Slider;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

import javax.imageio.ImageIO;
import java.awt.image.BufferedImage;
import java.io.*;
import java.net.ServerSocket;
import java.net.Socket;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.Statement;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
```

```

public class BlurServer extends Application {

    private volatile int currentKernelSize = 3;

    @Override
    public void start(Stage primaryStage) {
        // Ad 3. Interfejs z etykietą i suwakiem dla wartości nieparzystych 1-15
        Label label = new Label("Promień filtra (nieparzyste 1-15): " + currentKernelSize);

        Slider slider = new Slider(1, 15, currentKernelSize);
        slider.setMajorTickUnit(2);
        slider.setMinorTickCount(0);
        slider.setSnapToTicks(true);
        slider.setShowTickLabels(true);
        slider.setShowTickMarks(true);

        slider.valueProperty().addListener((obs, oldVal, newVal) -> {
            int val = newVal.intValue();
            if (val % 2 == 0) val += 1; // Wymuszenie wartości nieparzystych
            if (val > 15) val = 15;
            if (val != newVal.intValue()) {
                slider.setValue(val);
            }
            currentKernelSize = val;
            label.setText("Promień filtra (nieparzyste 1-15): " + val);
        });

        VBox root = new VBox(15, label, slider);
        root.setPadding(new Insets(20));

        Scene scene = new Scene(root, 350, 150);
        primaryStage.setTitle("Box Blur Server");
        primaryStage.setScene(scene);
        primaryStage.setOnCloseRequest(e -> System.exit(0));
        primaryStage.show();

        startServerThread();
    }

    private void startServerThread() {
        Thread serverThread = new Thread(() -> {
            try (ServerSocket serverSocket = new ServerSocket(5000)) {
                System.out.println("Serwer oczekuje na klientów (port 5000)...");
                while (true) {
                    // Serwer blokuje się na akceptacji, obsługując 1 klienta naraz
                    try (Socket clientSocket = serverSocket.accept()) {
                        System.out.println("Obsługa nowego klienta...");
                        handleClient(clientSocket);
                    } catch (Exception e) {
                        e.printStackTrace();
                    }
                }
            }
        });
    }
}

```

```

        }
    } catch (IOException e) {
        e.printStackTrace();
    }
});
serverThread.setDaemon(true);
serverThread.start();
}

private void handleClient(Socket socket) throws Exception {
    DataInputStream input = new DataInputStream(socket.getInputStream());
    DataOutputStream output = new DataOutputStream(socket.getOutputStream());

    // 1 & 2. Otrzymanie i zapis pliku PNG od klienta
    long fileSize = input.readLong();

    File imagesDir = new File("images");
    if (!imagesDir.exists()) {
        imagesDir.mkdir();
    }

    String timestamp = LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyy-MM-dd_HH-mm-ss"));
    String filename = timestamp + ".png";
    File savedFile = new File(imagesDir, filename);

    try (FileOutputStream fos = new FileOutputStream(savedFile)) {
        byte[] buffer = new byte[8192];
        long receivedSize = 0;
        int count;
        while (receivedSize < fileSize) {
            int toRead = (int) Math.min(buffer.length, fileSize - receivedSize);
            count = input.read(buffer, 0, toRead);
            if (count == -1) break;
            fos.write(buffer, 0, count);
            receivedSize += count;
        }
    }

    // 3. Odczytanie promienia / rozmiaru jądra z interfejsu GUI
    int kernelSize = currentKernelSize;

    // 4. Przekształcenie obrazu algorytmem box blur zrównoleglonym
    BufferedImage image = ImageIO.read(savedFile);
    if (image == null) throw new Exception("Błędny plik graficzny.");

    long startTime = System.currentTimeMillis();
    BufferedImage blurredImage = applyBoxBlur(image, kernelSize);
    long delay = System.currentTimeMillis() - startTime;

    File blurredFile = new File(imagesDir, timestamp + "_blurred.png");
    ImageIO.write(blurredImage, "png", blurredFile);
}

```

```

// 5. Zapis do bazy danych SQLite db
saveToDatabase(savedFile.getPath(), kernelSize, delay);

// 6. Odesłanie przetworzonego pliku do klienta
byte[] blurredBytes = java.nio.file.Files.readAllBytes(blurredFile.toPath());
output.writeLong(blurredBytes.length);
output.write(blurredBytes);
output.flush();

System.out.println("Przetworzono i odesłano obraz do klienta.");
}

// Box blur używający podziału na pulę wątków równej liczbie rdzeni
private BufferedImage applyBoxBlur(BufferedImage image, int kernelSize) throws Exception {
    if (kernelSize <= 1) return image;

    int width = image.getWidth();
    int height = image.getHeight();
    int[] pixels = image.getRGB(0, 0, width, height, null, 0, width);
    int[] outPixels = new int[pixels.length];

    int cores = Runtime.getRuntime().availableProcessors();
    ExecutorService executor = Executors.newFixedThreadPool(cores);

    int r = kernelSize / 2;
    int chunkHeight = height / cores;

    for (int i = 0; i < cores; i++) {
        final int startY = i * chunkHeight;
        final int endY = (i == cores - 1) ? height : (i + 1) * chunkHeight;

        executor.submit(() -> {
            for (int y = startY; y < endY; y++) {
                for (int x = 0; x < width; x++) {
                    int sumR = 0, sumG = 0, sumB = 0, sumA = 0;
                    int count = 0;

                    for (int ky = -r; ky <= r; ky++) {
                        int ny = y + ky;
                        if (ny < 0 || ny >= height) continue;

                        for (int kx = -r; kx <= r; kx++) {
                            int nx = x + kx;
                            if (nx < 0 || nx >= width) continue;

                            int argb = pixels[ny * width + nx];
                            sumA += (argb >> 24) & 0xFF;
                            sumR += (argb >> 16) & 0xFF;
                            sumG += (argb >> 8) & 0xFF;
                            sumB += argb & 0xFF;

```

```

        count++;
    }
}

    int avgA = sumA / count;
    int avgR = sumR / count;
    int avgG = sumG / count;
    int avgB = sumB / count;

    outPixels[y * width + x] = (avgA << 24) | (avgR << 16) | (avgG << 8) |
}
}
});
}

executor.shutdown();
try {
    executor.awaitTermination(1, TimeUnit.HOURS);
} catch (InterruptedException e) {
    e.printStackTrace();
}

BufferedImage outImage = new BufferedImage(width, height, BufferedImage.TYPE_INT_ARGB);
outImage.setRGB(0, 0, width, height, outPixels, 0, width);
return outImage;
}

private synchronized void saveToDatabase(String path, int size, long delay) {
    String url = "jdbc:sqlite:images/index.db";
    try (Connection conn = DriverManager.getConnection(url)) {
        try (Statement stmt = conn.createStatement()) {
            stmt.execute("CREATE TABLE IF NOT EXISTS images (" +
                "id INTEGER PRIMARY KEY AUTOINCREMENT, " +
                "path TEXT, " +
                "size INTEGER, " +
                "delay INTEGER)");
        }
        String sql = "INSERT INTO images (path, size, delay) VALUES (?, ?, ?)";
        try (PreparedStatement pstmt = conn.prepareStatement(sql)) {
            pstmt.setString(1, path);
            pstmt.setInt(2, size);
            pstmt.setLong(3, delay);
            pstmt.executeUpdate();
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}

public static void main(String[] args) {
    launch(args);
}

```

```
}
```

```
}
```